

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

з Лабораторної роботи 1

з дисципліни «Архітектура програмного забезпечення»

на тему «Vision and Scope»

Виконав:

ст. гр. ПЗП-23-3

Ромасенко С.О.

Прийняв:

ст. викл. Сокорчук І.П.

Харків 2026

1 ІСТОРІЯ ЗМІН

№	Дата	Версія звіту	Опис
1	2.05.2026	0.1	Створено розділ “Завдання”
2	2.05.2026	0.1	Створено розділ “Опис виконаної роботи” та “Висновок”
3	3.05.2026	0.1	Створено Додаток А,Б

2 ЗАВДАННЯ

1. Доопрацювати опис проєкту у форматі Vision & Scope, зосередившись на деталізації рамок проєкту (Scope).
2. Розширити розділ "Scope and Limitations", додавши опис початкового релізу з урахуванням IoT-технологій та план майбутніх релізів з використанням ШІ.
3. Обґрунтувати вибір методів збору даних та навчання моделей машинного навчання для оптимізації системи бронювання.
4. Створити звіт до лабораторної роботи, експортувати його у формат PDF та завантажити на платформу dl.nure.ua.
5. Забезпечити збереження звіту у текстовому форматі (UTF-8) у відповідному GitHub репозиторії.

3 ОПИС ВИКОНАНОЇ РОБОТИ

У межах лабораторної роботи було проведено глибоку модернізацію документа Vision & Scope для системи RoomBook. Основну увагу приділено розділу Scope and Limitations, де було деталізовано архітектуру початкового релізу, що тепер включає не лише Backend та Frontend (Web/PWA), а й IoT-компоненти для автоматизації контролю присутності в приміщеннях.

Окремо розроблено стратегію впровадження штучного інтелекту в наступних ітераціях проєкту. Визначено три ключові аспекти: використання ШІ для інтелектуальних рекомендацій та прогнозування, методи збору аналітичних даних про використання ресурсів, а також підхід до використання готових API та донавчання (fine-tuning) моделей на основі накопиченої статистики. Також було критично переглянуто межі проєкту, виділено неочевидні обмеження та обґрунтовано виключення певного функціоналу для збереження стабільності системи.

Використання засобів ШІ:

Назва інструменту та версія: Gemini 3 Flash.

Мета використання: Структурування оновлених розділів Scope, генерація ідей для інтеграції IoT та ШІ, а також стилістичне редагування звіту.

Внесок автора: Формулювання бізнес-логіки системи, визначення пріоритетів функціональних модулів (MF-1 — MF-4) та адаптація згенерованих пропозицій під специфіку проєкту RoomBook.

4 ВИСНОВКИ

Доопрацювання документа Vision & Scope дозволило вийти за межі базового функціоналу та спроектувати сучасну інтелектуальну систему управління ресурсами. Впровадження IoT та планування використання ШІ забезпечує RoomBook конкурентними перевагами: автоматизацією боротьби з "No-Show" та можливістю предиктивного аналізу попиту.

Чітке окреслення рамок наступних релізів мінімізує ризики розмивання скоупу та дозволяє команді розробників фокусуватися на найбільш критичних бізнес-вимогах, гарантуючи масштабованість та технологічну актуальність рішення у довгостроковій перспективі.

Додаток А

Відеозапис

Посилання на відеозапис доповіді: <https://youtu.be/xMMzolUqA28>

Хронологічний опис відеозапису:

00:00 - Вступ

00:20 - Пояснення про розділ

00:37 - Scope and Initial release

01:12 - Scope of subsequent releases

01:51 - Limitations and Exclusions

02:20 - Висновок

Додаток Б

Vision & Scope

Vision and Scope Document

**for
Room Booking Software System**

Version 1.0 approved

Prepared by Stanislav Romasenko

Nure

11.12.2025

TABLE OF CONTENTS

Revision History 7

1 BUSINESS REQUIREMENTS 8

 1.1 Background & Business Opportunities 8

 1.2 Customer and Market Needs 9

2 VISION OF THE SOLUTION 10

 2.1 Vision Statement 10

 2.2 Business Logic & Workflow 10

 2.2.1 MF-1: Real-Time Availability & Booking 10

 2.2.2 MF-2: Booking Management & Notifications 10

 2.2.3 MF-3: Administrative Room Management 11

 2.2.4 MF-4: Usage Analytics & Reporting 11

 2.2.5 MF-5: IoT-Driven Access Control & Automated Presence Verification 12

3 SCOPE AND LIMITATIONS Ошибка! Закладка не определена.

 3.1 Scope: Backend & Database Ошибка! Закладка не определена.

 3.2 Scope: Frontend (Web & Mobile) Ошибка! Закладка не определена.

 3.3 Hardware Requirements & IoT Integration (IoT-клієнт) Ошибка! Закладка не определена.

 3.4 Hardware Limitations Ошибка! Закладка не определена.

 4.1 Technology Stack 15

Revision History

Name	Date	Reason for Changes	Version
First version	11.12.2025	Initial creation	1.0

1 BUSINESS REQUIREMENTS

1.1 Background & Business Opportunities

Проблема неоптимального використання спільних фізичних ресурсів (наприклад, навчальних аудиторій, переговорних кімнат, спеціалізованих лабораторій або проєктних офісів) є поширеною у великих організаціях та навчальних закладах. Традиційні методи бронювання, які включають паперові журнали, загальні електронні календарі (без централізованої перевірки конфліктів) або запити електронною поштою, регулярно призводять до кількох ключових проблем: подвійне бронювання, неефективне використання (коли заброньована кімната простоє) та відсутність достовірних даних для планування інвестицій у приміщення.

Програмна система RoomBook створюється для централізації та автоматизації цього процесу. Вона має на меті перетворити хаотичний процес резервування на прозорий, керований механізм. Основна бізнес-можливість полягає в підвищенні коефіцієнта використання приміщень (Utilization Rate) до 70-80% у пікові години за рахунок надання користувачам можливості швидкого пошуку та бронювання доступних слотів. Для адміністрації система відкриває можливість отримувати точну аналітику щодо попиту на конкретні типи кімнат та часу доби, дозволяючи обґрунтовано коригувати графіки, перерозподіляти ресурси та планувати майбутнє розширення приміщень. Це, у свою чергу, призводить до зменшення операційних витрат та підвищення загальної задоволеності користувачів.

1.2 Customer and Market Needs

Існуючі універсальні рішення, такі як загальні календарні служби (Google Calendar, Outlook), не повністю задовольняють потреби у бронюванні фізичних ресурсів, оскільки вони не мають спеціалізованих функцій:

1. Потреба у фільтрації за атрибутами приміщення: Користувачам потрібно швидко знайти кімнату не просто за часом, а й за наявністю специфічного обладнання (наприклад, 3D-принтер, інтерактивна дошка, місткість 15+ осіб).
2. Потреба у запобіганні конфліктам: Універсальні календарі можуть дозволити подвійне бронювання, якщо користувач не має прав редагування, або якщо приміщення помилково додано до кількох незалежних календарів. RoomBook забезпечує атомарність бронювання.
3. Потреба в аналітиці "No-Show": Адміністрації необхідно відстежувати, які бронювання були проігноровані, щоб запровадити політику штрафів або автоматичного звільнення слотів.
4. Потреба в інтеграції з корпоративним/університетським обліком: Необхідна інтеграція із системою управління користувачами (наприклад, Active Directory або LDAP) для забезпечення єдиної точки входу (SSO) та автоматичного присвоєння ролей.

Система RoomBook вирішує ці проблеми, надаючи централізований реєстр ресурсів із деталізованими атрибутами та інтуїтивно зрозумілий інтерфейс. Вона фокусується на потребах кінцевого користувача в простоті та швидкості бронювання, а також на потребах адміністратора в контролі та глибокій аналітиці використання ресурсів.

2 VISION OF THE SOLUTION

2.1 Vision Statement

RoomBook — це хмарна, модульна програмна система на базі .NET, яка слугує єдиною платформою для ефективного управління та бронювання спільних фізичних ресурсів. Мета системи – забезпечити прозорість графіків, мінімізувати конфлікти бронювання та надати адміністративному персоналу інструменти для прийняття рішень щодо оптимізації використання приміщень. Ми прагнемо створити надійний, швидкий та масштабований сервіс, що стане еталоном для внутрішніх систем управління ресурсами.

2.2 Business Logic & Workflow

Бізнес-логіка системи реалізується через взаємодію основних функціональних модулів (MF) на сервері, які керують даними та обробляють запити від веб- та мобільного клієнтів.

2.2.1 MF-1: Real-Time Availability & Booking

Цей модуль є серцем системи. Користувач взаємодіє з інтерактивним календарем або списком, де відображається доступність кімнат. Запит на бронювання включає: час початку/закінчення, обрану кімнату, кількість учасників та мету бронювання. Система миттєво перевіряє у базі даних, чи не існує конфлікту (подвійного бронювання, перевищення максимальної місткості, бронювання поза робочими годинами кімнати). У разі успіху, бронювання фіксується атомарною транзакцією. Специфікація: Система повинна відповідати на запит доступності кімнат протягом не більше 500 мс, навіть при високому навантаженні. Введені дані повинні бути валідовані на стороні клієнта та сервера.

2.2.2 MF-2: Booking Management & Notifications

Модуль надає користувачам можливість переглядати список їхніх майбутніх та минулих бронювань. Користувач може змінити час або скасувати

активне бронювання до певного моменту (наприклад, за 5 хвилин до початку). Система генерує автоматичні сповіщення:

- Підтвердження бронювання (миттєво).
- Нагадування (за 1 годину до початку).
- Запит на підтвердження присутності (наприклад, за 10 хвилин до початку), що є основою для функції No-Show (див. нижче).
- Сповіщення про скасування (якщо адміністратор скасував).

2.2.3 MF-3: Administrative Room Management

Модуль для бізнес-адміністраторів, який забезпечує повний контроль над реєстром ресурсів. Функціональність включає:

- Створення, редагування, архівування приміщень.
- Визначення атрибутів: місткість, робочі години (наприклад, лабораторія працює лише з 9:00 до 18:00), наявність обладнання (проектор, відеоконференц-зв'язок).
- Управління користувачами та ролями: Призначення адміністраторів, модераторів та звичайних користувачів.
- Ручне втручання: Можливість для адміністратора примусово скасувати або змінити будь-яке бронювання з обов'язковим надсиланням пояснювального сповіщення ініціатору.

2.2.4 MF-4: Usage Analytics & Reporting

Цей модуль накопичує та обробляє історичні дані про бронювання для формування аналітичних звітів, необхідних для оптимізації ресурсів. Ключові показники, які обчислюються:

- Коефіцієнт використання (Utilization Rate): Відсоток фактично заброньованого часу від загального доступного часу для кожної кімнати та/або категорії.
- Рівень "No-Show": Відсоток бронювань, які були проігноровані або не підтверджені. Це дозволяє ідентифікувати проблемних користувачів або кімнати з надмірним бронюванням.

- Піковий попит: Візуалізація найпопулярніших годин, днів тижня та типів кімнат, що може слугувати підставою для динамічного ціноутворення або відкриття додаткових приміщень у майбутньому.
- Звітність: Система повинна дозволяти експорт основних звітів у формати XLSX та PDF з можливістю фільтрації за періодом та категорією приміщень.

2.2.5 MF-5: IoT-Driven Access Control & Automated Presence Verification

Цей модуль розширює функціональність системи за межі програмного забезпечення, інтегруючи її з апаратними засобами контролю доступу для повної автоматизації процесу використання приміщень.

- Автоматичне керування фізичним доступом: Система виступає як центральний контролер для IoT-пристроїв (електронних замків), встановлених у приміщеннях. У момент настання часу валідного бронювання (StartTime) сервер автоматично ініціює команду розблокування через WebSocket-з'єднання. Це мінімізує потребу у фізичній передачі ключів або присутності охоронця.
- Real-Time взаємодія через SignalR: Завдяки використанню технології SignalR, обмін даними між сервером та IoT-клієнтом відбувається миттєво. Специфікація: затримка передачі команди від сервера до апаратного модуля не повинна перевищувати 200 мс, що забезпечує комфортний користувацький досвід.
- Автоматизація політики "No-Show": Модуль IoT є основним джерелом даних для аналітики присутності. Якщо IoT-датчик (або зчитувач доступу) не фіксує активності протягом встановленого «вікна очікування» (наприклад, 15 хвилин від початку бронювання), система автоматично скасовує запис у БД, надсилає сповіщення (MF-2) та змінює статус кімнати на «Доступна» (MF-1).
- Безпека та авторизація пристроїв: Кожен апаратний пристрій ідентифікується за унікальним IotDeviceId та проходить процедуру захищеної авторизації. Це виключає можливість несанкціонованого доступу до керування замками приміщень сторонніми особами або шкідливим ПЗ.

3 SCOPE AND LIMITATIONS

3.1 Scope of Initial Release

Початковий реліз системи RoomBook спрямований на створення цілісної екосистеми для управління фізичними ресурсами. Вона охоплює:

- **Серверна частина (Backend):** Високопродуктивний RESTful API на базі ASP.NET Core 8, що забезпечує атомарність транзакцій при бронюванні та інтеграцію з MS SQL Server.
- **Web-клієнт:** Повнофункціональний SPA-застосунок (Vue.js 3) для адміністраторів та користувачів, що дозволяє керувати реєстром кімнат, переглядати аналітику та здійснювати візуальне бронювання.
- **Мобільний доступ:** Реалізація у форматі PWA, що забезпечує крос-платформність (iOS/Android) та підтримку push-сповіщень про підтвердження або скасування броні.
- **IoT-компонент:** Інтеграція з розумними замками або датчиками присутності для автоматичної фіксації факту використання кімнати. Це вирішує проблему «No-Show», автоматично звільняючи слот, якщо в кімнаті нікого немає протягом 10–15 хвилин після початку сесії.

3.2 Scope of Subsequent Releases (AI Integration)

У наступних версіях планується впровадження штучного інтелекту за трьома напрямками:

1. Застосування (AI Use Cases):

- **Інтелектуальні рекомендації:** ШІ аналізуватиме потреби користувача (кількість людей, тип обладнання) і пропонуватиме оптимальну кімнату або альтернативний час у разі конфлікту.
- **Предиктивне обслуговування:** Прогнозування виходу з ладу обладнання (наприклад, ламп у проєкторах) на основі інтенсивності використання.

2. Збирання даних:

- Система накопичуватиме історичні логи: час бронювання vs реальний час використання (через IoT-датчики), популярність конкретних характеристик кімнат, сезонні піки навантаження.
- Дані збиратимуться анонімно через модуль аналітики та зберігатимуться в Data Lake для подальшого аналізу.

3. Навчання та використання:

- Для загальних задач (чат-бот підтримки) використовуватиметься готовий API (наприклад, OpenAI).
- Для специфічних задач (прогнозування попиту) планується донавчання (Fine-tuning) існуючих моделей регресії на власних зібраних даних, що дозволить адаптувати алгоритми під специфіку конкретної організації.

3.3 Limitations and Exclusions

- Обмеження: Система вимагає постійного інтернет-з'єднання; офлайн-бронювання не підтримується через ризик конфліктів даних.
- Виключення: Навмисно виключено пряму інтеграцію з фінансовими білінговими системами та складні моделі прогнозування на базі ML у першій фазі, щоб уникнути надмірної складності архітектури та зосередитись на стабільності базового функціоналу.

4 ASSUMPTIONS AND DEPENDENCIES

4.1 Technology Stack

Технологічний стек проекту RoomBook сформовано з урахуванням вимог до створення високопродуктивної, надійної та масштабованої системи. В архітектурі домінує екосистема .NET (C#), що забезпечує узгодженість серверної частини, спрощує процеси розробки та підтримує крос-платформне розгортання. Для клієнтської частини та допоміжної інфраструктури обрано сучасні, індустріально-стандартні інструменти, які мінімізують складність та максимізують ефективність роботи.

Компонент	Технологія	Обґрунтування вибору
Backend/API	C# / ASP.NET Core 8	Вибір C# та .NET Core забезпечує високу продуктивність, надійність та крос-платформну підтримку. ASP.NET Core є одним з найшвидших фреймворків для створення RESTful API.
Database	Microsoft SQL Server	Використання надійної реляційної СУБД, яка добре інтегрується з екосистемою Microsoft (.NET/C#).
ORM	Entity Framework Core	Сучасний, гнучкий та потужний ORM для .NET, що спрощує взаємодію з SQL Server та підтримує концепцію Code-First.
Caching / Pub-Sub	Redis	Використання Redis як in-memory сховища для швидкого доступу до часто запитуваних даних (наприклад, кешування доступності кімнат) та як брокер для легкого обміну повідомленнями/подій.
Frontend (Web/PWA)	Vue.js 3 / TypeScript	Сучасний JavaScript-фреймворк, який, на відміну від React, забезпечує високу швидкість розробки та легку криву навчання. Підтримує розробку PWA.
Asynchronous Tasks	Hangfire	Бібліотека для .NET, яка дозволяє легко створювати фонові та заплановані завдання (наприклад, щоденна генерація звітів, надсилання нічних сповіщень) без необхідності впровадження важкого message broker.
Deployment	Docker	Контейнеризація всіх компонентів (.NET API, SQL Server, Redis, Vue.js Nginx-сервер) для забезпечення

		консистентності середовищ розробки, тестування та продакшену.
Source Control	Git / GitHub	Стандартний для галузі механізм контролю версій та співпраці.

Залежності:

1. Доступ до SMTP-сервера: Для коректної роботи системи сповіщень необхідний доступ до зовнішнього або внутрішнього SMTP-сервера.
2. Інфраструктура розгортання: Необхідна наявність віртуальної машини (Linux або Windows Server) або хмарного середовища (Azure, AWS) з підтримкою Docker та доступом до бази даних SQL Server.
3. Політики організації: Розробка залежить від чітких політик щодо робочих годин приміщень, правил бронювання та прав доступу користувачів, які мають бути надані адміністративним персоналом до початку імплементації.